

type. As a consequence, the Numpy arrays use much less memory than regular lists. Also, most of the Numpy operations is implemented in the C language, meaning, the cost of Python loops and dynamic checking of the data type is avoided. This, yields a significant increase in processing speed when comparing Numpy to a Python list.

More often than one we encounter large datasets with tens of thousands rows of data, just think of hourly air temperature measurements for a county or region since the measurements beginning in this region. If your weather service is measuring hourly air temperature for last 50 years, that's more that 400 000 rows of data, just for one station.

How to install Numpy?

Well, if using Anaconda, Numpy is preinstalled in the base environment. However, more often than not, it's good practice to create new environments for new projects. To install Numpy, we run an Anaconda prompt and type:

```
| conda install numpy
```

or

```
| conda install -c anaconda numpy
```

If pip is being used, Numpy can be installed by typing:

```
| pip install numpy
```

How to import Numpy?

When importing certain libraries, including Numpy, we follow a convention, basically this means we use well established abbreviations for libraries. In the case of Numpy we use "np".

```
| import numpy as np
```

The goal is that our code is reproducible, and every Python programmer in the World, knows what the following line does:

```
| a = np.array([3,4])
```

Congrats, if you have imported Numpy, and used the above command, you have successfully created your first **Numpy array**. Let's see what happens if we print it out. Print gives us something that looks like a list, but it's not. When we check the type we see that's a "**numpy.ndarray**" (*n dimensional array*).

```
In [9]: a = np.array([3,4])
        print (a)
        print (type(a))
```

